西安電子科技大學

《计算机操作系统课程设计》

课 程 报 告

题　　目 实验三：Priority-Scheduling—Task1

指导教师 李海昆

学　　院 计算机科学与技术学院　专　　业 计算机科学与技术

学　　号 19050500032　班　　级 1903011

姓　　名 许珈铭

完成时间 2021 年 12 月 23 日

目录

1 实验目的 **1**

2 实验原理 **1**

2.1 pintos 系统内核代码理解 1

2.2 基本原理 4

3 实验方案 **6**

3.1 环境配置 6

3.2 正式实验 10

3.3 实验结果 15

4 实验心得 **15**

实验三：Priority-Scheduling—Task1

1　实验目的

1. 了解 pintos 操作系统对于线程的调度采用的是最为简单的 FCFS（先来先服务）策略，没有考虑优先级问题。
2. 理解 threads.c、threads.h 文件中有关线程的优先级的结构体、函数的含义与作用
3. 要求为 Pintos 建立优先级调度机制，确保任何时刻 CPU 上运行的都是最高优先级线程。

2　实验原理

2.1　pintos 系统内核代码理解

1. *../pintos/src/threads/* 目录下的 *thread.h* 和 *thread.c* 文件是有关线程初始化、阻塞、解除阻塞，线程调度等内容。
   可以通过输入 `geany thread.c` `geany thread.h` 命令用 *geany* 文本编辑器来打开相应的文件查看内容。当然也可以使用 *vim* 或 *gedit* 文本编辑器来替换 *geany*。

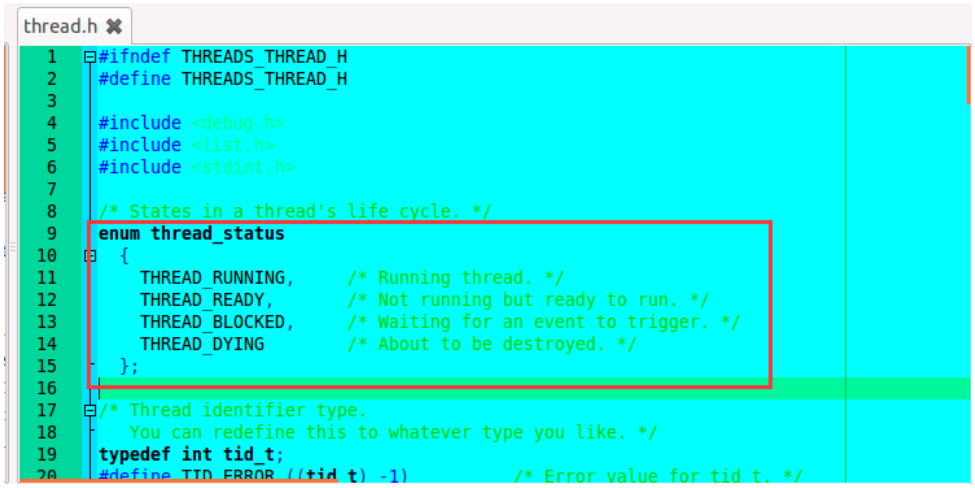

图 1: thread.h 文件

- 理解 *thread.h* 文件中的 *PRI_MIN*、*PRI_DEFAULT*、*PRI_MAX* 宏定义。

```c
struct thread
  {
    /* Owned by thread.c. */
    tid_t tid;                          /* Thread identifier. */
    enum thread_status status;          /* Thread state. */
    char name[16];                      /* Name (for debugging purposes). */
    uint8_t *stack;                     /* Saved stack pointer. */
    int priority;                       /* Priority. */
    struct list_elem allelem;           /* List element for all threads list. */

    /* Shared between thread.c and synch.c. */
    struct list_elem elem;              /* List element. */

#ifdef USERPROG
    /* Owned by userprog/process.c. */
    uint32_t *pagedir;                  /* Page directory. */
#endif

    /* Owned by thread.c. */
    unsigned magic;                     /* Detects stack overflow. */
  };
```

图 2: thread 结构体内容

tid	该线程的编号
thread_status	该线程目前的状态（enum 类型）
name[16]	该线程的名字
priority	该线程的优先级
...	...

其中 *priority* 代表的是该进程的优先级，在 *thread.h* 文件中已经定义了线程优先级的范围，如下图所示：

```c
/* Thread priorities. */
#define PRI_MIN 0                       /* Lowest priority. */
#define PRI_DEFAULT 31                  /* Default priority. */
#define PRI_MAX 63                      /* Highest priority. */
```

图 3: 线程优先级范围

- *thread.h* 和 *thread.c* 文件中还有一些需要了解其作用的函数，如下表：

*thread_create(const char *name,int priority,thread_func *function,void *aux)*	生成线程
thread_set_priority(int new_priority)	设置新的线程优先级
thread_current	获得当前正在运行的进程编号
...	...

2. 何时优先级会对调度产生影响？

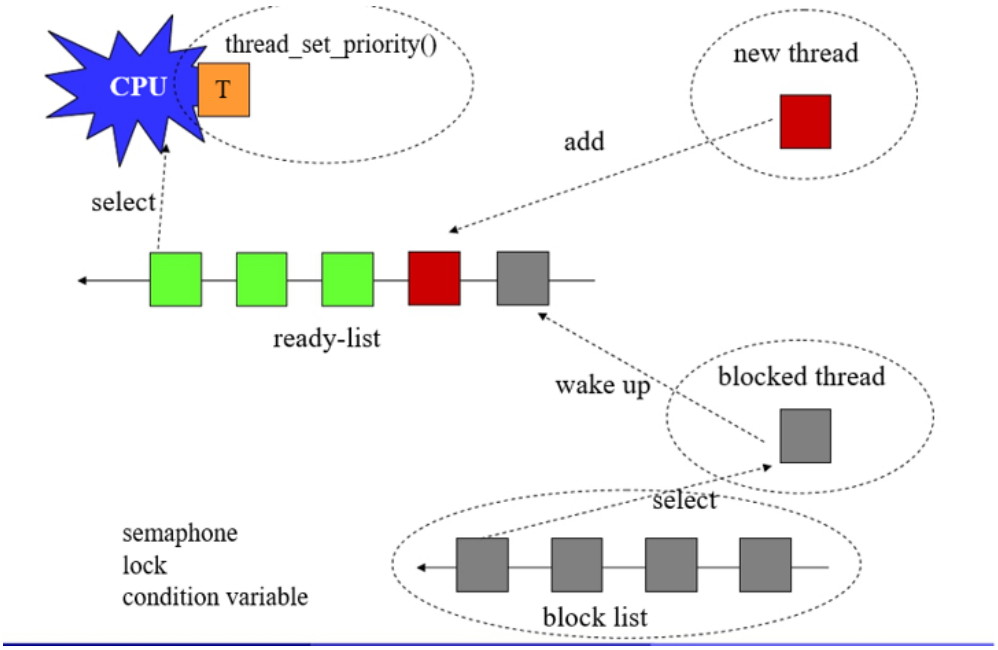

图 4: 存在优先级比较的线程调度流程

从图中可以一目了然，当有新的进程生成的时候，会初始化带有一定的优先级(*priority*)，那么需要和当前正在执行的线程和 *ready_list* 中的线程进行优先级大小比较，决定当前线程是否让出 CPU 交给新的进程。

3. *../pintos/src/devices/* 目录下的 *thread.c* 文件中包含了线程生成和调度所需要的函数。由上图的线程优先级比较图可知一个线程生成时就会自带一个优先级，所以线程创建函数 *thread_create()* 的参数中一定会有优先级参数, 线程创建完成后调用线程初始化函数，也应该要将优先级传入到初始化函数中，同时在线程调度时有可能会对线程的优先级进行修改，所以一定有优先级设置函数 *thread_set_priority()* 参数中，具体函数内容如下表：

*thread_create(const char *name, int priority,thread_func *function ,void *aux)*	线程创建函数
*init_thread(struct thread *t, const char *name, int priority)*	线程初始化函数
thread_set_priority	线程优先级设置函数
...	...

2.2 基本原理

1. 回顾存在优先级比较的线程调度流程图, 并思考线程生成与调度流程

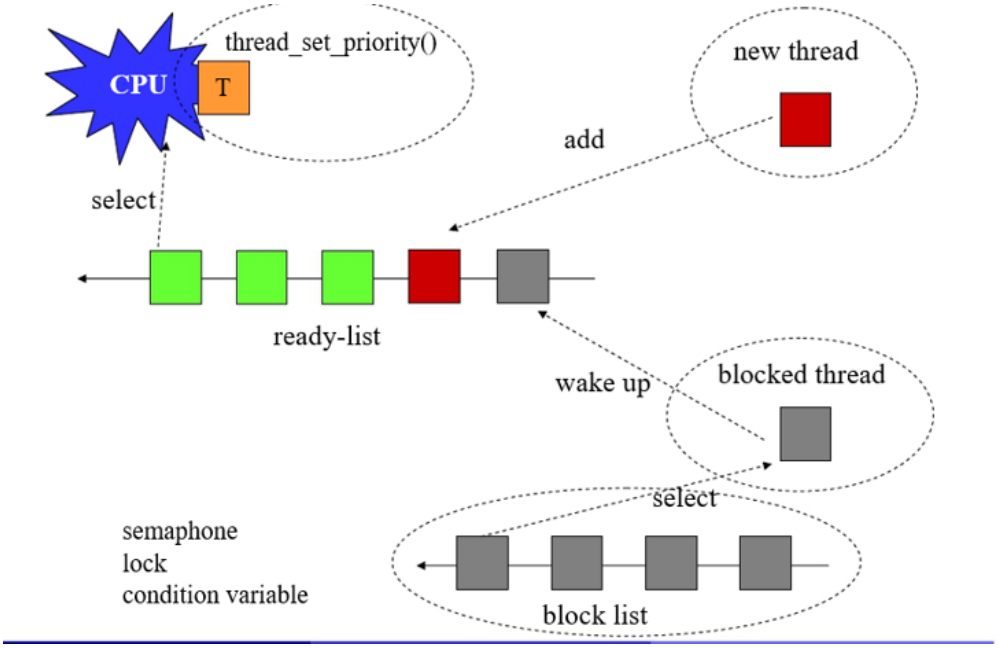

图 5: 存在优先级比较的线程调度流程

- 线程在生成时利用线程创建函数 *thread_create()* 就会自身带有初始优先级。
- 在线程创建函数执行后会主动调用线程初始化函数 *init_thread()* 对创建好的 *thread* 对象进行初始化，这里传入参数有优先级 *priority*。初始化完成后，当前线程会有 *ready-list* 中的线程优先级进行比较，确定当前线程是否让出 CPU，不让的话则将该线程放入 *ready-list* 的队尾，对于唤醒了的阻塞的进程也会放入 *ready-list* 当中。
- 对于新创建的进程没有获取到 *CPU* 后，需要执行 *thread_yield()* 函数插入了到 *ready_list*，对于阻塞的进程，可以执行 *thread_unblock()* 函数从 *block-list* 中唤醒进程进入到 *ready-list*。具体程序实现如下：

```
void
thread_yield (void)
{
  struct thread *cur = thread_current ();
  enum intr_level old_level;

  ASSERT (!intr_context ());

  old_level = intr_disable ();
  if (cur != idle_thread)
    list_push_back (&ready_list, &cur->elem);
  cur->status = THREAD_READY;
  schedule ();
  intr_set_level (old_level);
}
```

图 6: *thread_yield()* 函数

```
void
thread_unblock (struct thread *t)
{
  enum intr_level old_level;

  ASSERT (is_thread (t));

  old_level = intr_disable ();
  ASSERT (t->status == THREAD_BLOCKED);
  list_push_back (&ready_list, &t->elem);
  t->status = THREAD_READY;
  intr_set_level (old_level);
}
```

图 7: *thread_unblock()* 函数

2. 当 *ready-list* 接收到新的线程或者被唤醒的阻塞的进程，会进行有序化处理，具体实现和程序如下图：

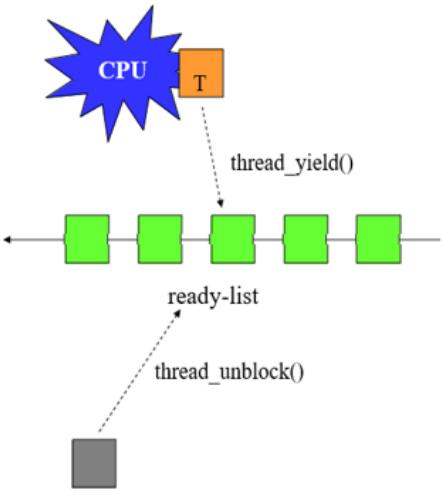

图 8: 基本原理图

```
void
list_insert_ordered (struct list *list, struct list_elem *elem,
                     list_less_func *less, void *aux)
{
  struct list_elem *e;

  ASSERT (list != NULL);
  ASSERT (elem != NULL);
  ASSERT (less != NULL);

  for (e = list_begin (list); e != list_end (list); e = list_next (e))
    if (less (elem, e, aux))
      break;
  return list_insert (e, elem);
}
```

图 9: *list_insert_ordered()* 函数

- 当执行就绪队列的有序化过程中，需要进行线程同步机制，所以要用到信号量，那么对应的就会存在信号量的 PV 操作，在这里主要由函数 *sema_down()* 和函数 *sema_up()* 负责执行。同时还需要解锁和开锁临界区的定义。具体细节如下图

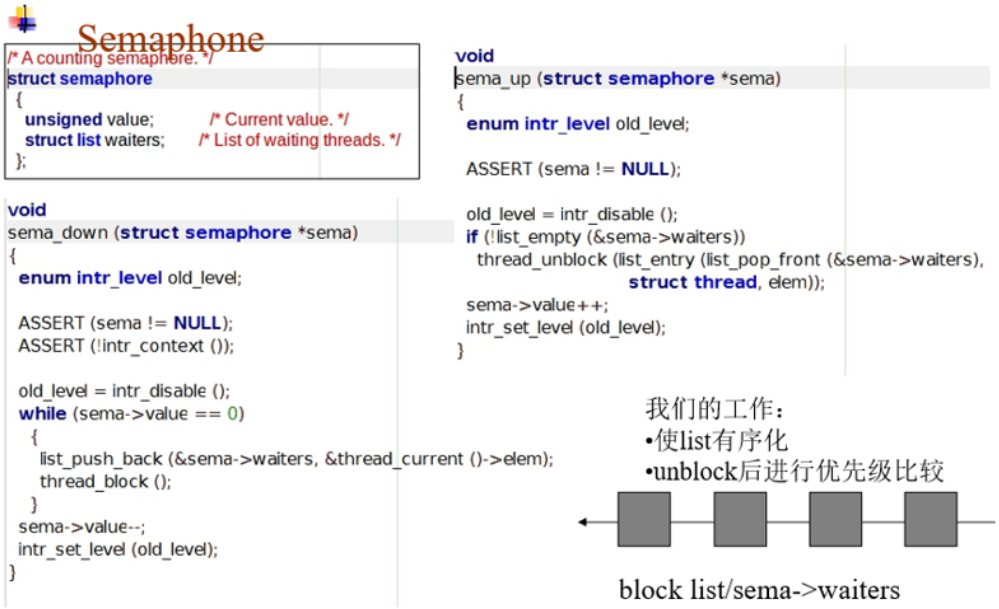

图 10: 线程同步机制

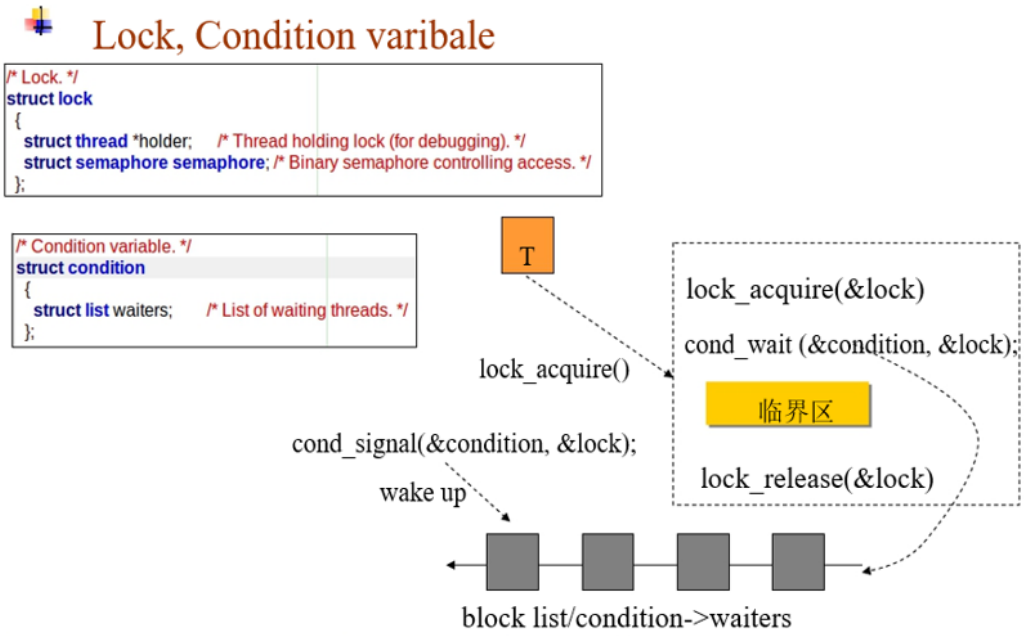

图 11: 线程同步机制

3　实验方案

3.1　环境配置

3.1.1　云服务器的连接

- 实验工具：阿里云服务器、Xshell 远程连接软件、VNC Viewer。
- 实验步骤：按如下步骤执行即可

  1. 打开 Xshell 软件，点击新建

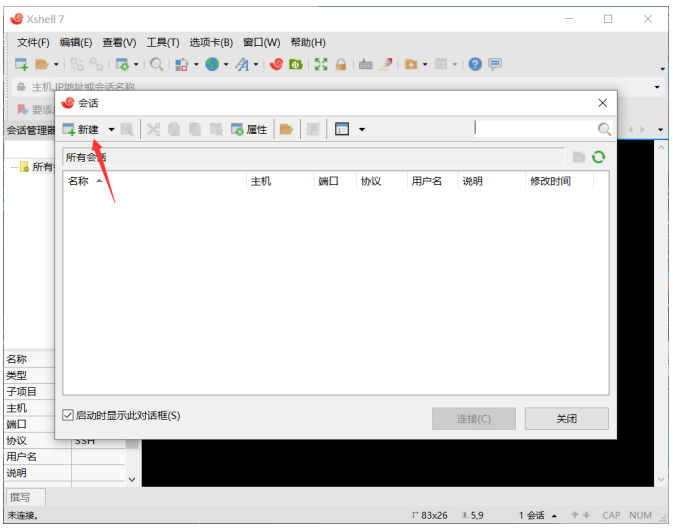

图 12: Xshell 软件界面-1

2. 在阿里云网站上申请相应的服务器资源后，按照图中指示顺序填写相关内容

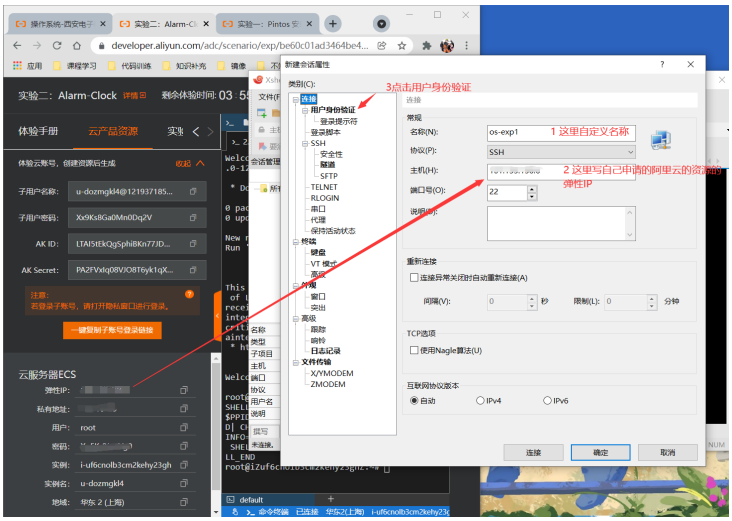

图 13: Xshell 软件界面-2

3. 然后点击用户身份验证，填入相应的身份表示信息

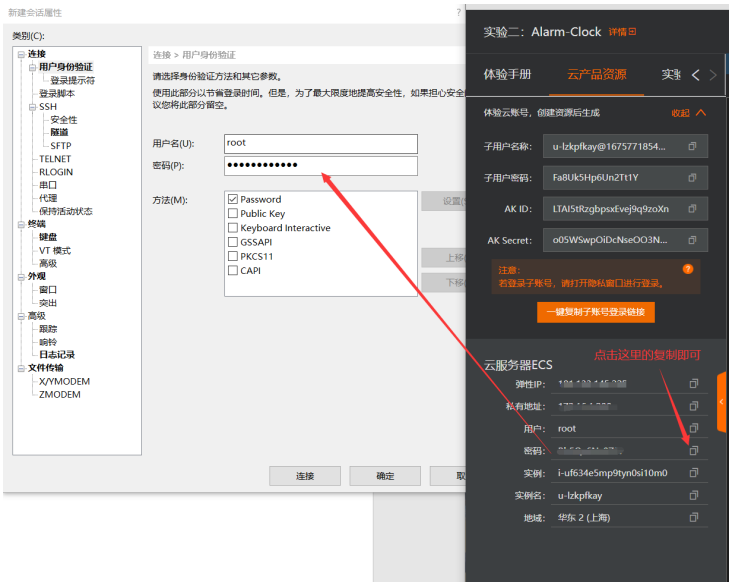

图 14: Xshell 软件界面-3

4. 出现这样的界面点击“接受并保存”，出现 *Welcome to aliyun Elastic Compute Sevice* 则表示远程连接成功了。

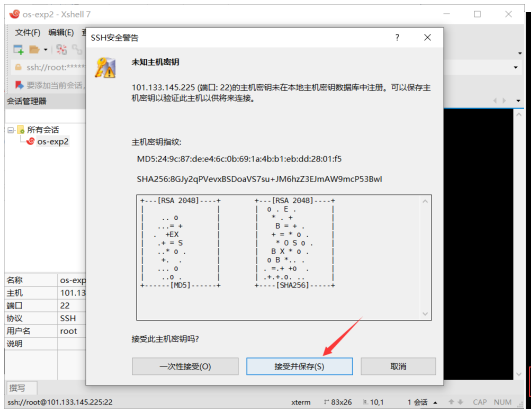

图 15: Xshell 软件界面-4

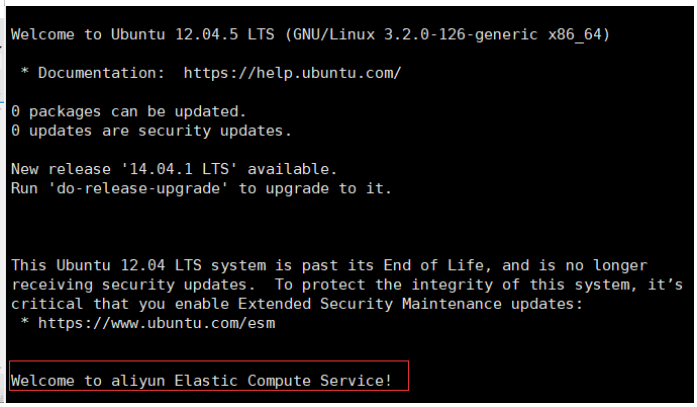

图 16: Xshell 软件界面-5

3.1.2　用 *VNC Viewer* 使用图形化界面登录

- 实验工具：*VNC Viewer*。
- 实验步骤：按如下步骤执行即可

1. 安装 *VNC Viewer* 后，打开软件，在相应的地址栏输入**弹性 IP 加上:1**

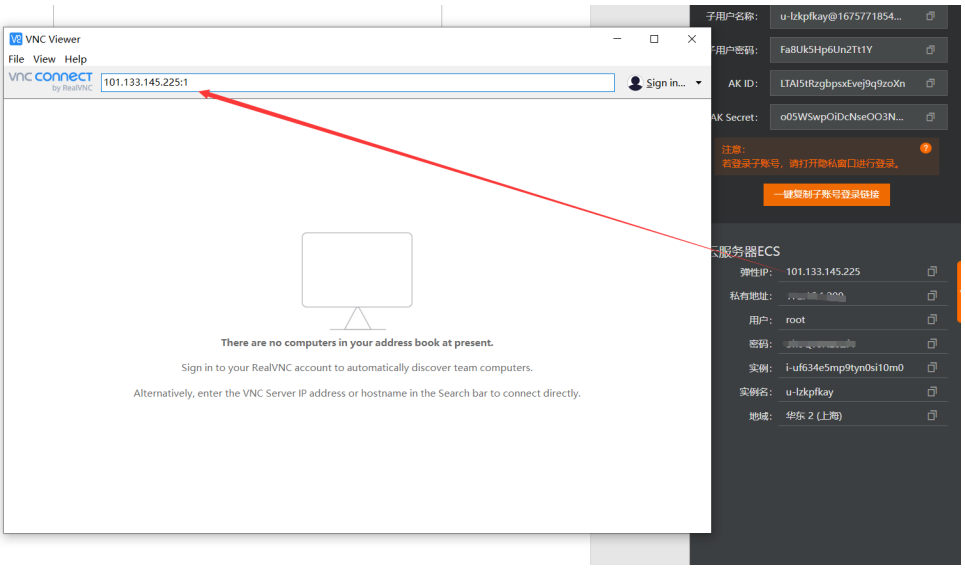

图 17: VNC 软件界面-1

2. 如果出现如下左图，直接点击 *continue* 即可显示右图，输入初始密码 *123456*

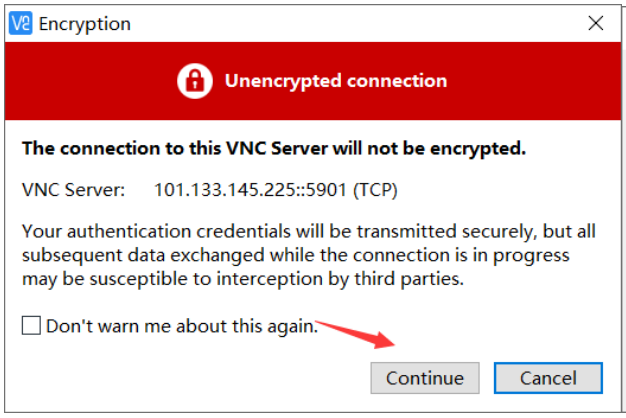

图 18: VNC 软件界面-2

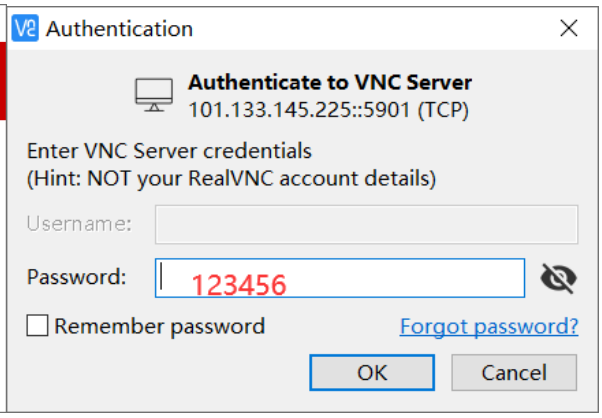

图 19: VNC 软件界面-3

3. 如果出现下图，一般这样的情况都是所连接的服务器没有配 *VNC* 的环境。

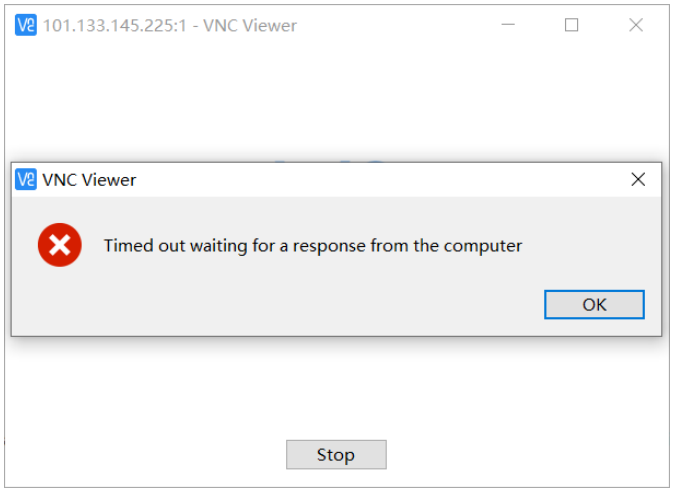

图 20: VNC 软件界面-4

所以需要先用 Xshell 按照 3.1.1 的云服务器的连接步骤连接服务器（Windows 用 Powershell 中的 SSH 也行，Mac 直接 Terminal 中 SSH 命令，本质都是一样的）进入命令行模式。在 *Xshell* 的命令行中依次输入:

- `vncserver :1`
- `vncserver -geometry 2560x1440`

注意:这里的‘x’是字母 x，不是‘*’，这里是调整分辨率，可以根据你的电脑显示屏的分辨率来调，可以低于电脑分辨率，不能高于分辨率，我这里的显示器是 2k 屏，所以是 2560x1440,1080P 的话应该就是 1920x1080 不然可能会出现颜色乱码。

注意:结束实验后，服务器会自动清除相关配置，所以在每一次体验创建资源后，都需要配置一遍 *VNC Viewer*，不然就会出现上图的 *Timed out*

4. 我们成功进入图形化界面, 我们的基本配置也算是结束了。

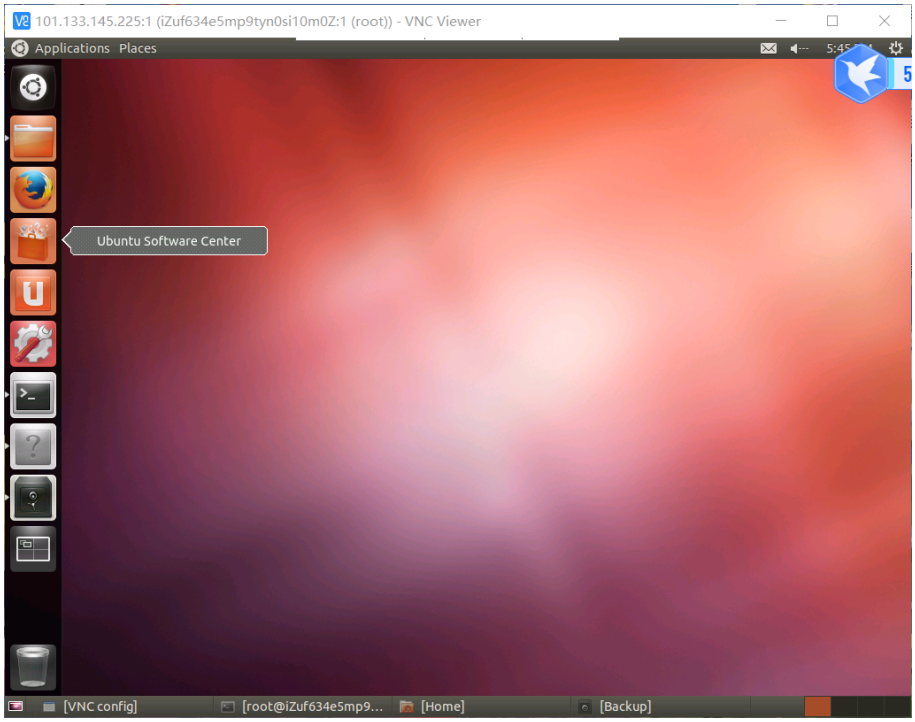

图 21: VNC 软件界面-5

注意：因为这是在云服务器上完成的，所以服务器已经给我们预装了 *bochs* 虚拟机以及安装在其中的 *pintos* 系统。可以在对应的*/home* 下文件夹下查看。如果没有下图的内容，则需要自己安装 *bochs* 和 *pintos*。

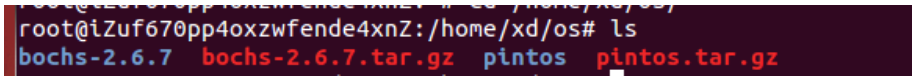

图 22: VNC 软件界面-6

3.2　正式实验

根据实验目的，基于对实验内容的充分理解的基础上，我们需要修改一下三个文件中的代码。

文件名	路径
thread.h	*/pintos/src/threads*
synch.c	*/pintos/src/threads*
thread.c	*/pintos/src/threads*
synch.h	*/pintos/src/threads*
...	...

1. 在 *thread.c* 中修改 thread_create() 函数

```
    /* Add to run queue */
    thread_unblock(t);
    /* 添加 */
        if (thread_current()->priority<t->priority)
            thread_yield();
    /* 结束 */
    return tid;
```

2. 在 *thread.c* 中修改 *thread_yield()*

```
void thread_yield (void)
{
    struct thread *cur = thread_current ();
    enum intr_level old_level;
    ASSERT (!intr_context ());
    old_level = intr_disable ();
    if (cur != idle_thread)
    /* 删除 */
    // list_push_back (&ready_list, &cur->elem);
    /* 加入 */
    list_insert_ordered(&ready_list,&cur->elem,thread_less,NULL);
    /* 结束 */
    cur->status = THREAD_READY;
    schedule ();
    intr_set_level (old_level);
}
```

3. 在 *thread.c* 中修改 *thread_unblock()*

```
void thread_unblock (struct thread *t)
{
    enum intr_level old_level;
    ASSERT (is_thread (t));
    old_level = intr_disable ();
    ASSERT (t->status == THREAD_BLOCKED);
    /* 删除 */
    //list_push_back (&ready_list, &t->elem);
    /* 添加 */
    list_insert_ordered(&ready_list,&t->elem,thread_less,NULL);
    /* 结束 */
    t->status = THREAD_READY;
    intr_set_level (old_level);
}
```

4. 在 *thread.c* 中修改 *thread_set_priority()*

```c
void thread_set_priority (int new_priority)
{
    thread_current ()->priority = new_priority;
    /* 添加 */
    if(!list_empty(&ready_list))
    {
        if(new_priority <= list_entry(list_begin(&ready_list),
            struct thread,elem)->priority)
        {
            thread_yield();
        }
    }
    /* 结束 */
}
```

5. 在 *thread.c* 和 *thread.h* 中对 *thread_less()* 函数和 *thread_priority_check()* 进行定义：

Step 1: 将 *thread_less()* 和 *thread_priority_check()* 函数加入到 *thread.h* 中

```c
bool thread_less(struct list_elem *elem1,struct list_elem *
    elem2,void *aux);
void thread_priority_check(void);
```

Step 2: 在 *thread.c* 中定义 *thread_less()* 和 *thread_priority_check()* 函数

```c
/* 添加 */
bool thread_less(struct list_elem *elem1,struct list_elem *
    elem2,void *aux)
{
    struct thread *t1,*t2;
    t1=list_entry(elem1,struct thread,elem);
    t2=list_entry(elem2,struct thread,elem);
    return (t1->priority>t2->priority);
}

void thread_priority_check()
{
    struct thread *t =thread_current();
    struct list_elem *top=list_begin(&ready_list);
    struct thread *ready = list_entry (top,struct thread,elem
        );
    if(t != NULL &&t->priority <ready->priority)
    {
        thread_yield();
    }
}
/* 结束 */
```

6. 在 *synch.c* 中修改函数 *sema_down()*

```c
void sema_down (struct semaphore *sema)
{
    enum intr_level old_level;
    ASSERT (sema != NULL);
    ASSERT (!intr_context ());
    old_level = intr_disable ();
    while (sema->value == 0)
    {
        /* 删除 */
        // list_push_back (&sema->waiters, &thread_current ()->elem);
        /* 添加 */
        list_insert_ordered(&sema->waiters,&thread_current ()->elem,thread_less,NULL);
        /* 结束 */
        thread_block ();
    }
    sema->value--;
    intr_set_level (old_level);
}
```

7. 在 *synch.c* 中修改函数 *sema_up()*

```c
void sema_up (struct semaphore *sema)
{
    enum intr_level old_level;
    ASSERT (sema != NULL);
    old_level = intr_disable ();
    if (!list_empty (&sema->waiters))
        thread_unblock (list_entry (list_pop_front (&sema->waiters),struct thread, elem));
    sema->value++;
    intr_set_level (old_level);
    /* 添加 */
    thread_priority_check();
    /* 添加 */
}
```

8. 在 *synch.c* 中修改结构体 *semaphore_elem*

```c
struct semaphore_elem
{
    struct list_elem elem; /* List element. */
    struct semaphore semaphore; /* This semaphore. */
    /* 添加 */
    int priority;
```

```
    /* 结束 */
};
```

9. 在 *synch.c* 中修改函数 *cond_wait()*

```
void cond_wait (struct condition *cond, struct lock *lock)
{
    struct semaphore_elem waiter;
    ASSERT (cond != NULL);
    ASSERT (lock != NULL);
    ASSERT (!intr_context ());
    ASSERT (lock_held_by_current_thread (lock));
    sema_init (&waiter.semaphore, 0);
    /* 删除 */
    //list_push_back (&cond->waiters, &waiter.elem);
    /* 添加 */
    waiter.priority = thread_current()->priority;
    list_insert_ordered(&cond->waiters, &waiter.elem,cond_less,NULL
        );
    /* 结束 */
    lock_release (lock);
    sema_down (&waiter.semaphore);
    lock_acquire (lock);
}
```

10. 在 *synch.c* 和 *synch.h* 中对 *cond_less()* 函数进行定义：

Step 1: 将 *cond_less()* 函数加入到 *synch.h* 中

```
int cond_less(struct list_elem *elem1,struct list_elem *elem2
    );
```

Step 2: 在 *synch.c* 中定义 *cond_less()* 函数

```
/* 添加 */
int cond_less(struct list_elem *elem1,struct list_elem *elem2
    )
{
    struct semaphore_elem *s1,*s2;
    s1 = list_entry(elem1,struct semaphore_elem,elem);
    s2 = list_entry(elem2,struct semaphore_elem,elem);
    return s1->priority > s2->priority;
}
/* 结束 */
```

3.3　实验结果

在 *teriminal* 中进入*../pintos/src/threads/*目录，输入：

```
make check
```

观察到只有以下 13 个检测均通过，其余均为不通过则为实验成功！如下图所示

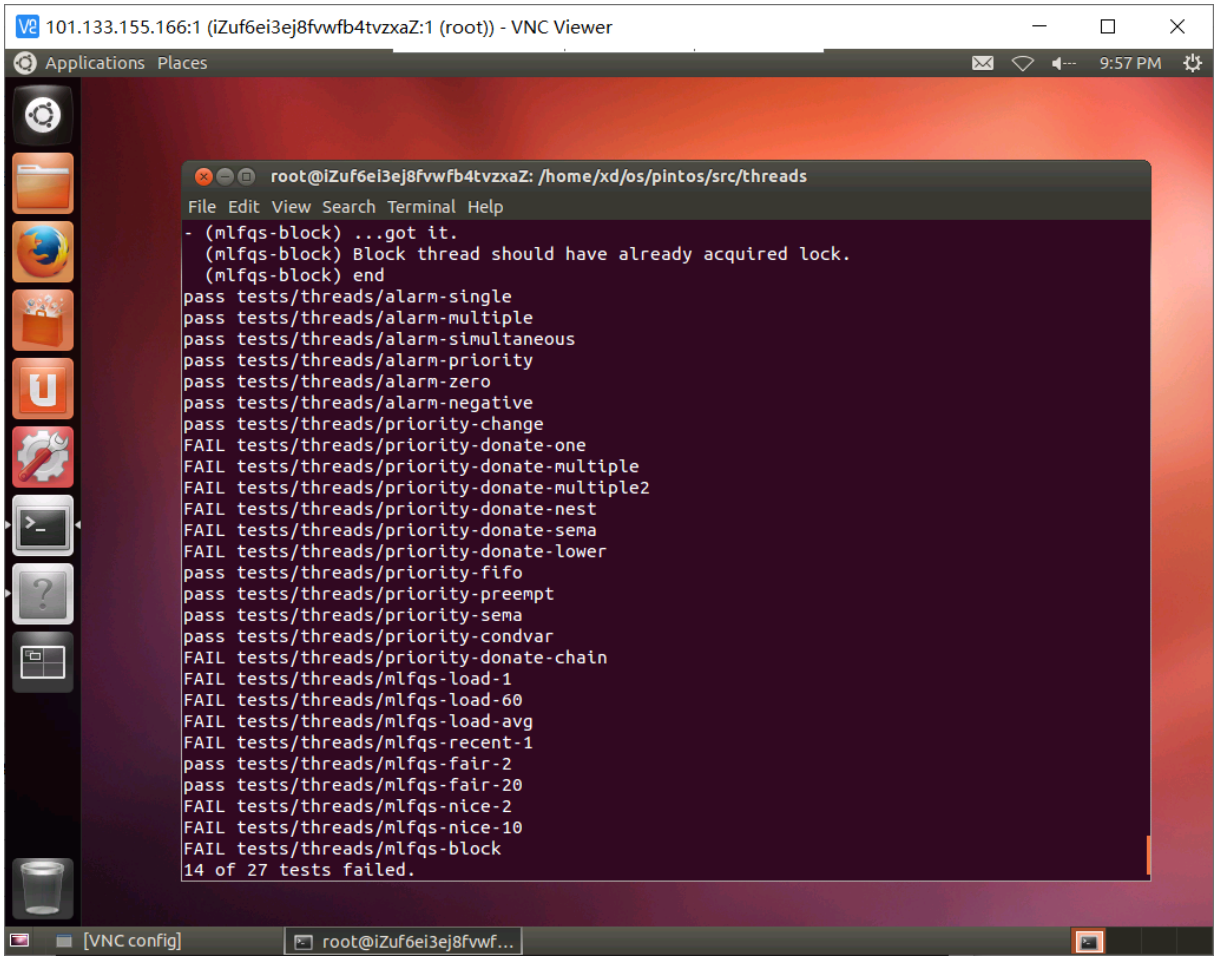

图 23: 结果截图

4　实验心得

通过本次实验，我粗略地了解了 *pintos* 的简单内核代码的含义，了解了有关线程优先级的定义和线程工作之间的调度关系，本次实验是通过给线程添加优先级属性，并且在线程创建、就绪队列的有序化、线程调度等情况的时候考虑优先级的作用，不再是 FCFS 先来先服务地执行线程。当执行该函数时，则将该进程阻塞，很显然这样做法也会导致某些进程无法进行正常工作。